fcm_push_kit

Reusable Firebase push notifications for Flutter (Android and iOS)
Proof of concept and integration guide

Item	Value
Package	`packages/fcm_push_kit` (version 0.1.0)
Repository	github.com/Soumya8792/push_notification
Demo app	`push_notification_demo` (same repository)
Firebase project	`push-notification-bfe0b` (project number 845589201307)
Platforms	Android 7.0+ (API 24), iOS 15.0+
Stack	Flutter 3.47, firebase_core 5.1, firebase_messaging 17.0, flutter_local_notifications 22.3
Date	3 November 2026

1. Purpose and scope

Every app we build needs the same push notification plumbing: Firebase setup, permission prompts, device tokens, showing notifications while the app is open, and opening the right screen when a user taps one. This POC packages that plumbing once as **fcm_push_kit**, so a new project adds one dependency, does the native setup checklist, and calls one initialize method.

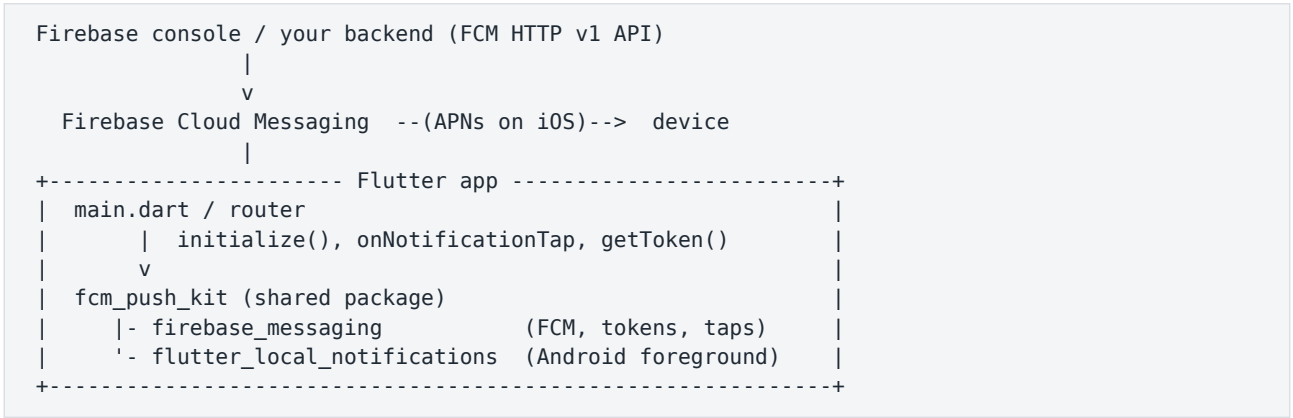
The POC proves the following on both Android and iOS:

- Notifications are received in all three app states: foreground, background and terminated.
- A tap opens the screen named in the payload (deep link), e.g. /details?id=42.
- Device tokens can be read and refreshed, and topics can be subscribed to.
- The same package can be consumed by several apps via a git dependency.

Out of scope for the POC: a production backend that stores tokens, notification analytics, rich media (images), action buttons, and web/desktop targets.

2. Architecture

The app talks only to fcm_push_kit. The package wraps the two Flutter plugins and hides the platform differences between Android and iOS.



Behaviour per app state

App state	Android	iOS	Tap reaches the app via
Foreground	Package shows a local notification on the high-importance channel	System banner via foreground presentation options (no duplicate)	`onNotificationTap` (source: foreground / background)
Background	System tray (FCM SDK)	System banner (APNs)	`onNotificationTap` (source: background)
Terminated	System tray (FCM SDK)	System banner (APNs)	`getInitialTap()` after the first frame

Repository layout

```
push_notification_demo/
  lib/main.dart                 <- initializes FcmPushKit, handles initial tap
  lib/router/app_router.dart    <- go_router + tap listener + "Copy FCM Token"
  android/app/build.gradle.kts  <- core library desugaring
  ios/Runner/AppDelegate.swift  <- UNUserNotificationCenter delegate
  ios/Runner/Runner.entitlements<- aps-environment (Push Notifications)
  ios/Runner/Info.plist         <- UIBackgroundModes: remote-notification
  packages/fcm_push_kit/        <- the reusable package
    lib/src/fcm_push_kit.dart          FcmPushKit (main API)
    lib/src/fcm_push_kit_config.dart   FcmPushKitConfig
    lib/src/push_notification_tap.dart PushNotificationTap, TapSource
    test/                              unit tests
```

3. Fixing the "Firebase CLI Login Failed" error

Symptom: after choosing the Google account, the browser shows "Oops! Firebase CLI Login Failed" on localhost:9005.

Cause (from firebase-debug.log): the browser step succeeded, but the CLI's follow-up request that exchanges the code for a token was cut off: `FetchError: Invalid response body ... oauth2/token: Premature close`. This is a dropped network connection (VPN, proxy, firewall or unstable Wi-Fi), seen here with an outdated CLI (15.22.0) on Node 24.

Fix:

1. Update the CLI (already done on this machine, now 15.34.2): `npm install -g firebase-tools@latest`
2. Start clean and log in again: `firebase logout` then `firebase login --reauth`. Pick the Google account that owns the project in the console.
3. If it still fails, disconnect VPN/proxy and retry, or use the copy-paste flow, which does not rely on localhost: `firebase login --no-localhost`
4. Last resort: use Node 22 LTS: `nvm install 22 && nvm use 22 && npm i -g firebase-tools`
5. Verify: `firebase projects:list` must list `push-notification-bfe0b`.

4. Firebase project setup (once per app)

The console currently shows "There are no apps in your project". FlutterFire CLI registers the Android and iOS apps and writes the config files for you.

1. Choose final app IDs first. The demo still uses `com.example.push_notification_demo` (Android) and `com.example.pushNotificationDemo` (iOS); change them to your own reverse-domain IDs before registering, because Firebase and APNs are tied to them.
2. Install FlutterFire CLI: `dart pub global activate flutterfire_cli`
3. From the app root run:

```
flutterfire configure \
  --project=push-notification-bfe0b \
  --platforms=android,ios
```

This creates `lib/firebase_options.dart`, `android/app/google-services.json`, `ios/Runner/GoogleService-Info.plist` and adds the Google Services Gradle plugin. Re-run it whenever app IDs change.

5. Adding the package to a project

Inside this repository

```
dependencies:
  firebase_core: ^5.1.0
  fcm_push_kit:
    path: packages/fcm_push_kit
```

From any other project (recommended for reuse)

```
dependencies:
  firebase_core: ^5.1.0
  fcm_push_kit:
    git:
      url: https://github.com/Soumya8792/push_notification.git
      path: packages/fcm_push_kit
      ref: fcm_push_kit-v0.1.0   # tag; use main only while developing
```

Pin a tag in production apps so a change to the package never reaches an app until that app bumps the ref. Then run `flutter pub get`.

6. Android native setup

All of the following is already applied in the demo app.

android/app/build.gradle.kts

```
android {
    compileOptions {
        isCoreLibraryDesugaringEnabled = true   // required by flutter_local_notifications
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    defaultConfig {
        minSdk = flutter.minSdkVersion           // 24 in Flutter 3.47; 24 is required
    }
}

dependencies {
    coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.1.4")
}
```

android/app/src/main/AndroidManifest.xml

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

<application ...>
    <!-- must equal FcmPushKitConfig.androidChannelId -->
    <meta-data
        android:name="com.google.firebase.messaging.default_notification_channel_id"
        android:value="high_importance_channel" />
    <meta-data
        android:name="com.google.firebase.messaging.default_notification_icon"
        android:resource="@mipmap/ic_launcher" />
</application>
```

Android 13+ asks the user for notification permission; the package triggers the prompt during initialize() unless `requestPermissionOnInit: false`. For production, add a white, transparent small icon (e.g. `@drawable/ic_notification`) and use it in both the manifest and the config.

7. iOS native setup

Apple Developer account and Firebase console (manual, once per app)

1. developer.apple.com, then Certificates, IDs & Profiles, then Keys: create a key with **Apple Push Notifications service (APNs)** enabled and download the .p8 file (downloadable only once). Note the Key ID and your Team ID.
2. Firebase console, then Project settings, then Cloud Messaging, then Apple app configuration: upload the .p8 with the Key ID and Team ID. One key works for all apps in the team, for both development and production.
3. Make sure the App ID (bundle ID) has the Push Notifications capability.

Xcode project (already applied in the demo)

- **Push Notifications** capability: `ios/Runner/Runner.entitlements` with `aps-environment`, linked via `CODE_SIGN_ENTITLEMENTS`.
- **Background Modes**: `UIBackgroundModes` = fetch, remote-notification in Info.plist.
- Open `ios/Runner.xcworkspace` in Xcode, then Signing & Capabilities: select your Team. The two capabilities above should be listed.

ios/Runner/AppDelegate.swift

```
override func application(
  _ application: UIApplication,
  didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
) -> Bool {
  UNUserNotificationCenter.current().delegate = self as? UNUserNotificationCenterDelegate
  return super.application(application, didFinishLaunchingWithOptions: launchOptions)
}
```

Test iOS push on a **real iPhone**; Simulator support for APNs is limited. The FCM token on iOS is only available after the APNs token arrives; `getToken()` waits up to 5 seconds for it.

8. Using the package in Dart

Initialize in main()

```
import 'package:fcm_push_kit/fcm_push_kit.dart';
import 'package:flutter/material.dart';
import 'firebase_options.dart';

@pragma('vm:entry-point')
Future<void> backgroundMessageHandler(RemoteMessage message) async {
  await fcmPushKitDefaultBackgroundHandler(message); // initializes Firebase
  // e.g. update a badge count or local cache. No UI work here.
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await FcmPushKit.instance.initialize(
    firebaseOptions: DefaultFirebaseOptions.currentPlatform,
    config: const FcmPushKitConfig(
      topics: ['all'],
      backgroundHandler: backgroundMessageHandler,
      enableLogs: true,
    ),
  );

  // Taps while the app is running (foreground or background).
  FcmPushKit.instance.onNotificationTap.listen((tap) {
    final location = tap.location;            // "/details?id=42"
    if (location != null) router.go(location);
  });

  // Keep the backend's copy of the token current.
  FcmPushKit.instance.onTokenRefresh.listen(api.saveDeviceToken);

  runApp(const MyApp());
}
```

Handle the tap that launched the app

```
@override
void initState() {
  super.initState();
  WidgetsBinding.instance.addPostFrameCallback((_) async {
    final tap = await FcmPushKit.instance.getInitialTap();
    final location = tap?.location;
    if (location != null) router.go(location);
  });
}
```

Token, topics and logout

```
final token = await FcmPushKit.instance.getToken();   // send to your backend
await FcmPushKit.instance.subscribeToTopic('offers');
await FcmPushKit.instance.unsubscribeFromTopic('offers');
await FcmPushKit.instance.deleteToken();              // on logout
```

9. API reference

Member	Description
`initialize({firebaseOptions, config})`	Initializes Firebase (if needed), FCM, the Android channel, listeners, permission and topics. Call once before runApp.
`requestPermission()`	Shows the OS prompt; returns true when allowed (authorized or provisional).
`getToken()`	Current FCM registration token, or null. On iOS waits for APNs first.
`onTokenRefresh`	Stream of new tokens. Send each one to the backend.
`deleteToken()`	Invalidates the token (logout).
`subscribeToTopic / unsubscribeFromTopic`	Topic messaging, e.g. "all", "offers".
`onNotificationTap`	Stream of PushNotificationTap for taps while the app runs.
`getInitialTap()`	The tap that launched the app from terminated state; returns it once.
`onForegroundMessage`	Stream of RemoteMessage received while the app is open.
`register / onRegistered / unregister`	Firebase Installation ID (FID) registration introduced in firebase_messaging 17, for projects moving away from tokens.
`isInitialized`	True after a successful initialize().

FcmPushKitConfig field	Default	Meaning
`androidChannelId`	`high_importance_channel`	Must match the manifest meta-data.
`androidChannelName / Description`	High Importance Notifications	Shown in Android app settings.
`androidSmallIcon`	`@mipmap/ic_launcher`	Small icon for foreground notifications.
`showForegroundNotifications`	true	Show banners while the app is open.
`requestPermissionOnInit`	true	Set false to ask later (after onboarding).
`routeKey`	`route`	Data key holding the deep-link route.
`topics`	[]	Topics to subscribe to on start.
`backgroundHandler`	built-in	Top-level function with @pragma('vm:entry-point').
`enableLogs`	false	Debug-build logs prefixed [FcmPushKit].

PushNotificationTap	Description
`data`	FCM data payload (all values are strings).
`source`	TapSource.foreground, background or terminated.
`route`	Value of data[routeKey], e.g. /details.
`location`	Route plus the other data keys as query parameters, e.g. /details?id=42.
`messageId`	FCM message ID when available.

10. Payload contract and sending notifications

Always send a **notification** block (title/body, so the OS shows it in the background) plus a **data** block (for navigation). Data values must be strings.

```
{
  "notification": { "title": "Order shipped", "body": "Tap to see details" },
  "data":         { "route": "/details", "id": "42" }
}
-> tap opens /details?id=42
```

Quick test from the Firebase console

1. Run the app on a device, press **Copy FCM Token** (also printed in the console as FCM TOKEN).
2. Firebase console, then Messaging, then Create your first campaign, then Firebase Notification messages. Enter a title and text.
3. Click **Send test message**, paste the token, then Test.
4. For deep links, under Additional options, then Custom data add `route = /details` and `id = 42` and publish the campaign to the app (or use the HTTP v1 request below).

From a backend: FCM HTTP v1 API

```
POST https://fcm.googleapis.com/v1/projects/push-notification-bfe0b/messages:send
Authorization: Bearer <OAuth2 access token of a service account>
Content-Type: application/json

{
  "message": {
    "token": "<device FCM token>",          // or "topic": "all"
    "notification": { "title": "Order shipped", "body": "Tap to see details" },
    "data": { "route": "/details", "id": "42" },
    "android": { "notification": { "channel_id": "high_importance_channel" } },
    "apns": { "payload": { "aps": { "sound": "default" } } }
  }
}
```

For a local test, an access token can be obtained with `gcloud auth print-access-token` (account with Firebase access). Backends should use the Firebase Admin SDK with a service account key from Project settings, then Service accounts. Never ship the service account key inside the app.

11. POC test checklist

#	Scenario	Expected result	Android	iOS
1	First launch	Permission prompt appears; token logged		
2	Foreground message	Banner shown once (no duplicate)		
3	Tap foreground banner	Opens /details?id=42		
4	Background message + tap	Tray/banner; tap opens target screen		
5	Terminated (swiped away) + tap	App starts on target screen		
6	Topic "all" message	Received without a token target		
7	Permission denied	App works; no notifications; no crash		
8	Reinstall app	New token issued (onTokenRefresh)		

12. Troubleshooting

Problem	Fix
"Firebase CLI Login Failed"	See section 3: update CLI, `firebase login --reauth`, avoid VPN/proxy, or `--no-localhost`.
"No Firebase App [DEFAULT] has been created"	Run `flutterfire configure`; check google-services.json / GoogleService-Info.plist exist.
No notification on Android 13+	Permission denied: Settings, then Apps, then Notifications. Check channel ID matches the manifest.
getToken() returns null on iOS	Real device needed; APNs key uploaded; Push Notifications capability; correct Team selected.
Background works, terminated tap does nothing	Call `getInitialTap()` after the first frame (not before runApp).
Duplicate banner on iOS	Do not show local notifications on iOS; the package already avoids this.
Android build error about desugaring	Add isCoreLibraryDesugaringEnabled and the desugar_jdk_libs dependency (section 6).
Force-stopped app gets nothing (Android)	Expected OS behaviour after Force stop; swiping away from recents is fine.

13. Rolling out to other projects

1. Tag a release of the package: `git tag fcm_push_kit-v0.1.0 && git push --tags`.
2. In the new app: add the git dependency (section 5) and `firebase_core`.
3. Run `flutterfire configure` against that app's Firebase project.
4. Apply the Android (section 6) and iOS (section 7) checklists.
5. Call `FcmPushKit.instance.initialize()` and wire taps to the app's router (section 8).
6. Run the test checklist (section 11) on a real Android phone and a real iPhone.

Changes to the package: bump `version` in its pubspec, update CHANGELOG.md, run `flutter test` in packages/fcm_push_kit, then tag. Apps upgrade by changing their ref.